

Modelli ricorsione TdP - NetworkX

Template da copiare nel Model per risolvere i secondi punti delle prove

Come usare questi modelli

- I template assumono che il grafo sia salvato in `self._graph`.
- Un cammino e una lista di nodi: `[n1, n2, n3]`. Il peso di un arco si legge con `self._graph[u][v]["weight"]`.
- Per grafo orientato usare `successors(n)`. Per grafo non orientato usare `neighbors(n)`.
- Per evitare cicli, ogni nodo già presente in parziale non viene riattraversato.
- Dove trovi `node.eta`, `node.dob`, `node.Essential`, `node.duration` devi adattare il nome al campo della tua dataclass.

Regola pratica da esame: prima scrivi il metodo pubblico che inizializza `self._bestPath`/`self._bestScore`, poi la funzione ricorsiva che prova tutti i vicini ammissibili.

Schema mentale generale

```
def cerca_soluzione(self):
    self._bestPath = []
    self._bestScore = -1

    for start in self._graph.nodes:
        parziale = [start]
        self._ricorsione(parziale)

    return self._bestPath, self._bestScore

def _ricorsione(self, parziale):
    # 1. Aggiorno la soluzione migliore
    if condizione_migliore(parziale, self._bestPath):
        self._bestPath = copy.deepcopy(parziale)

    ultimo = parziale[-1]

    # 2. Provo ad aggiungere un nodo vicino
    for vicino in self._graph.neighbors(ultimo):
        if vicino not in parziale and vincolo_valido(ultimo, vicino, parziale):
            parziale.append(vicino)
            self._ricorsione(parziale)
            parziale.pop()
```

Quale template usare?

Richiesta della traccia	Template	Esempi tipici
Cammino piu lungo con nodi non ripetuti	Template 1	eta decrescente, durata crescente
Cammino con archi a peso crescente/decrescente	Template 2 / 3	peso strettamente crescente o decrescente
Cammino da start a end con lunghezza fissa e peso massimo	Template 4	prodotti start/end + Lun
Cammino con punteggio personalizzato	Template 5	UFO: +100 nodo, +200 stesso mese
Scegliere K nodi/oggetti ottimi	Template 6 / 7	K circuiti/team, K piloti/costruttori

Template 0 - funzioni utili da tenere nel Model

```
import copy
import networkx as nx

def _peso_cammino(self, path):
    """Ritorna la somma dei pesi degli archi del cammino."""
    tot = 0
    for i in range(len(path)-1):
        u = path[i]
        v = path[i+1]
        tot += self._graph[u][v].get("weight", 1)
    return tot

def _vicini(self, nodo):
    """Usa successors se il grafo e diretto, altrimenti neighbors."""
    if self._graph.is_directed():
        return self._graph.successors(nodo)
    else:
        return self._graph.neighbors(nodo)

def _componenti(self):
    """Componenti corrette in base al tipo di grafo."""
    if self._graph.is_directed():
        return list(nx.weakly_connected_components(self._graph))
    else:
        return list(nx.connected_components(self._graph))
```

Template 1 - cammino piu lungo con vincolo sul nodo successivo

Usalo quando la traccia dice: cammino semplice di lunghezza massima e ogni nodo successivo deve avere un attributo crescente/decrecente. Esempio: eta strettamente decrescente.

```
def cerca_cammino_nodi_eta_decrecente(self):
    self._bestPath = []

    for start in self._graph.nodes:
        self._ricorsione_eta_decrecente([start])

    return self._bestPath

def _ricorsione_eta_decrecente(self, parziale):
    if len(parziale) > len(self._bestPath):
        self._bestPath = copy.deepcopy(parziale)

    ultimo = parziale[-1]

    for vicino in self._vicini(ultimo):
        if vicino in parziale:
            continue

        # ADATTA: cambia .eta con il campo corretto della tua dataclass
        if vicino.eta < ultimo.eta:
            parziale.append(vicino)
            self._ricorsione_eta_decrecente(parziale)
            parziale.pop()
```

Per durata crescente cambia la condizione in: if vicino.duration > ultimo.duration. Per data crescente: if vicino.date > ultimo.date.

Template 2 - cammino piu lungo con peso degli archi crescente

Usalo quando la traccia dice: ogni arco successivo deve avere peso strettamente crescente oppure crescente con $\geq$.

```
def cerca_cammino_pesi_crescenti(self):
    self._bestPath = []

    for start in self._graph.nodes:
        self._ricorsione_pesi_crescenti([start], None)

    return self._bestPath, self._peso_cammino(self._bestPath)

def _ricorsione_pesi_crescenti(self, parziale, ultimo_peso):
    # Criterio primario: cammino piu lungo
    # Criterio secondario: peso totale maggiore, se vuoi spareggiare
    if (len(parziale) > len(self._bestPath) or
        (len(parziale) == len(self._bestPath) and self._peso_cammino(parziale) >
         self._peso_cammino(self._bestPath))):
        self._bestPath = copy.deepcopy(parziale)

    ultimo = parziale[-1]

    for vicino in self._vicini(ultimo):
        if vicino in parziale:
            continue

        peso = self._graph[ultimo][vicino].get("weight", 1)

        # Per strettamente crescente: peso > ultimo_peso
        # Per crescente con uguale ammesso: peso >= ultimo_peso
        if ultimo_peso is None or peso > ultimo_peso:
            parziale.append(vicino)
            self._ricorsione_pesi_crescenti(parziale, peso)
            parziale.pop()
```

Template 3 - percorso di peso massimo con archi decrescenti da uno start

Usalo quando la traccia dice: vertice di partenza selezionato, peso degli archi strettamente decrescente, massimizzare il peso totale.

```
def cerca_percorso_peso_massimo_decrescente(self, start):
    self._bestPath = [start]
    self._bestWeight = 0

    self._ricorsione_peso_decrescente([start], None, 0)
    return self._bestPath, self._bestWeight

def _ricorsione_peso_decrescente(self, parziale, ultimo_peso, peso_totale):
    # Qui vince il peso totale massimo
    if peso_totale > self._bestWeight:
        self._bestWeight = peso_totale
        self._bestPath = copy.deepcopy(parziale)

    ultimo = parziale[-1]

    for vicino in self._vicini(ultimo):
        if vicino in parziale:
            continue

        peso = self._graph[ultimo][vicino].get("weight", 1)

        if ultimo_peso is None or peso < ultimo_peso:
            parziale.append(vicino)
            self._ricorsione_peso_decrescente(parziale, peso, peso_totale + peso)
```

```
parziale.pop()
```

Template 4 - start, end, lunghezza fissa, peso massimo

Usalo quando la traccia impone nodo iniziale, nodo finale, lunghezza Lun e somma pesi massima. Attenzione: qui Lun e il numero di nodi del cammino. Se nella tua traccia Lun indica numero di archi, usa `target_len = Lun + 1`.

```
def cerca_cammino_start_end_lunghezza(self, start, end, lun):
    self._bestPath = []
    self._bestWeight = -1

    self._ricorsione_start_end([start], end, lun, 0)
    return self._bestPath, self._bestWeight

def _ricorsione_start_end(self, parziale, end, lun, peso_totale):
    # Potatura: se il cammino e gia troppo lungo, stop
    if len(parziale) > lun:
        return

    ultimo = parziale[-1]

    # Caso base: lunghezza raggiunta
    if len(parziale) == lun:
        if ultimo == end and peso_totale > self._bestWeight:
            self._bestWeight = peso_totale
            self._bestPath = copy.deepcopy(parziale)
        return

    for vicino in self._vicini(ultimo):
        if vicino in parziale:
            continue

        peso = self._graph[ultimo][vicino].get("weight", 1)
        parziale.append(vicino)
        self._ricorsione_start_end(parziale, end, lun, peso_totale + peso)
        parziale.pop()
```

Template 5 - cammino con punteggio personalizzato e vincoli

Usalo per tracce tipo UFO: durata crescente, massimo 3 nodi dello stesso mese, +100 per nodo e +200 se stesso mese del precedente. Vale anche per punteggi simili.

```
def cerca_cammino_punteggio(self):
    self._bestPath = []
    self._bestScore = -1

    for start in self._graph.nodes:
        mesi = {start.datetime.month: 1} # ADATTA: campo data del nodo
        self._ricorsione_punteggio([start], 100, mesi)

    return self._bestPath, self._bestScore

def _ricorsione_punteggio(self, parziale, score, mesi_usati):
    if score > self._bestScore:
        self._bestScore = score
        self._bestPath = copy.deepcopy(parziale)

    ultimo = parziale[-1]

    for vicino in self._vicini(ultimo):
        if vicino in parziale:
            continue
```

```

# ADATTA: duration e datetime ai campi della tua dataclass
if vicino.duration <= ultimo.duration:
    continue

mese = vicino.datetime.month
if mesi_usati.get(mese, 0) >= 3:
    continue

punti = 100
if vicino.datetime.month == ultimo.datetime.month:
    punti += 200

mesi_usati[mese] = mesi_usati.get(mese, 0) + 1
parziale.append(vicino)
self._ricorsione_punteggio(parziale, score + punti, mesi_usati)
parziale.pop()
mesi_usati[mese] -= 1

```

Template 6 - scegliere K nodi minimizzando un range

Usalo quando la traccia chiede K piloti/costruttori in componenti connesse distinte e range minimo di date/eta/veterani. Prima calcoli la componente di ogni nodo, poi scegli combinazioni valide.

```

def cerca_k_min_range_componenti_distinte(self, k):
    # Mappa nodo -> indice componente
    comp_id = {}
    for i, comp in enumerate(self._componenti()):
        for nodo in comp:
            comp_id[nodo] = i

    # Tieni solo nodi con valore valido
    # ADATTA: usa .dob, .oldest_driver_dob, .eta, ecc.
    candidati = [n for n in self._graph.nodes if n.dob is not None]
    candidati.sort(key=lambda n: n.dob)

    self._bestSet = []
    self._bestRange = float("inf")

    self._ricorsione_k_range(candidati, k, 0, [], set(), comp_id)
    return self._bestSet, self._bestRange

def _ricorsione_k_range(self, candidati, k, indice, parziale, comp_usate, comp_id):
    if len(parziale) == k:
        valori = [n.dob for n in parziale] # ADATTA campo
        range_corrente = (max(valori) - min(valori)).days

        if range_corrente < self._bestRange:
            self._bestRange = range_corrente
            self._bestSet = copy.deepcopy(parziale)
        return

    if indice >= len(candidati):
        return

    # Potatura: non restano abbastanza candidati
    if len(parziale) + (len(candidati) - indice) < k:
        return

    nodo = candidati[indice]
    c = comp_id[nodo]

    # Scelta 1: prendo il nodo, solo se componente non ancora usata
    if c not in comp_usate:
        parziale.append(nodo)

```

```

        comp_usate.add(c)
        self._ricorsione_k_range(candidati, k, indice + 1, parziale, comp_usate, comp_id)
        comp_usate.remove(c)
        parziale.pop()

# Scelta 2: non prendo il nodo
self._ricorsione_k_range(candidati, k, indice + 1, parziale, comp_usate, comp_id)

```

Questo template è perfetto per: K piloti in componenti diverse con date di nascita più vicine; K costruttori in componenti diverse con veterani più vicini.

Template 7 - scegliere K elementi con punteggio massimo

Usalo per scegliere K circuiti/team tra candidati filtrati con soglia M, massimizzando la somma di un indice.

```

def cerca_k_migliori_per_score(self, k, m):
    candidati = []

    for nodo in self._graph.nodes:
        # ADATTA: condizione della traccia, es. almeno M edizioni/campionati
        if nodo.numero_edizioni >= m:
            candidati.append(nodo)

    self._bestSet = []
    self._bestScore = -1

    self._ricorsione_k_score(candidati, k, 0, [], 0)
    return self._bestSet, self._bestScore

def _ricorsione_k_score(self, candidati, k, indice, parziale, score):
    if len(parziale) == k:
        if score > self._bestScore:
            self._bestScore = score
            self._bestSet = copy.deepcopy(parziale)
        return

    if indice >= len(candidati):
        return

    if len(parziale) + (len(candidati) - indice) < k:
        return

    nodo = candidati[indice]

    # Scelta 1: prendo il nodo
    parziale.append(nodo)
    self._ricorsione_k_score(candidati, k, indice + 1, parziale, score + nodo.indice)
    parziale.pop()

    # Scelta 2: salto il nodo
    self._ricorsione_k_score(candidati, k, indice + 1, parziale, score)

```

Template 8 - cammino più lungo, a parità peso minimo

Usalo quando la traccia dice: identificare il cammino più lungo e, tra cammini di pari lunghezza, scegliere quello con peso totale minimo. Include esempio con Essential alternato e archi con peso crescente.

```

def cerca_cammino_lungo_peso_minimo(self):
    self._bestPath = []
    self._bestWeight = float("inf")

    for start in self._graph.nodes:
        self._ricorsione_lungo_minpeso([start], None, 0)

```

```

return self._bestPath, self._bestWeight

def _ricorsione_lungo_minpeso(self, parziale, ultimo_peso, peso_totale):
    if (len(parziale) > len(self._bestPath) or
        (len(parziale) == len(self._bestPath) and peso_totale < self._bestWeight)):
        self._bestPath = copy.deepcopy(parziale)
        self._bestWeight = peso_totale

    ultimo = parziale[-1]

    for vicino in self._vicini(ultimo):
        if vicino in parziale:
            continue

        peso = self._graph[ultimo][vicino].get("weight", 1)

        # Vincolo: archi solo crescenti con uguale ammesso
        if ultimo_peso is not None and peso < ultimo_peso:
            continue

        # Vincolo: due nodi consecutivi non possono avere stesso Essential
        # ADATTA: cambia .Essential con il campo corretto
        if vicino.Essential == ultimo.Essential:
            continue

        parziale.append(vicino)
        self._ricorsione_lungo_minpeso(parziale, peso, peso_totale + peso)
        parziale.pop()

```

Errori tipici da evitare

- Dimenticare `parziale.pop()`: dopo ogni chiamata ricorsiva devi tornare indietro.
- Usare `neighbors` su `DiGraph` quando devi rispettare il verso: meglio `successors`.
- Aggiornare la `best` solo nei nodi foglia: spesso devi aggiornarla a ogni chiamata, perché anche un cammino non estendibile potrebbe essere il migliore.
- Confondere lunghezza in nodi e lunghezza in archi: numero archi = `len(path)-1`.
- Non usare `copy.deepcopy`: se salvi `self._bestPath = parziale`, poi `parziale` cambia e perdi la soluzione.
- Non inizializzare `self._bestWeight`/`self._bestPath` nel metodo pubblico prima della ricorsione.

Mini-snippet per il Controller

```

# Esempio generico nel controller
path, score = self._model.cerca_cammino_pesi_crescenti()

self._view.txt_result.controls.append(ft.Text(f"Soluzione: {len(path)} nodi"))
self._view.txt_result.controls.append(ft.Text(f"Valore: {score}"))
for n in path:
    self._view.txt_result.controls.append(ft.Text(str(n)))
self._view.update_page()

```

Checklist prima di consegnare

- Il grafo è già stato creato? Se no, blocca la ricorsione.
- I dropdown start/end contengono oggetti nodo veri o stringhe? Se sono stringhe, devi recuperarli da `idMap`.
- Il grafo è orientato? Allora usa `successors`.
- La traccia massimizza lunghezza, peso, punteggio o minimizza range? Scegli il confronto corretto.
- Hai gestito caso nessuna soluzione? Stampa messaggio invece di fare crash.